

서사와 로드맵

"스킬 1개 = 클래스 1개" 안티패턴을 걷어낸 데이터 주도 전투 시스템 —
지금까지 한 것, 정직하게 남은 것, 그리고 어디로 갈지.

세 진영(플레이어·일반몹·보스)이 **제각각 하드코딩**하던 공격/스킬 로직을, **러너(AbilityRunner) + 데이터(AbilityData) + 실행 부품(IEffect)** 하나의 파이프라인으로 통합했다. **새 공격은 이제 코드가 아니라 데이터(Effect 조합)로 만든다.** 이 문서는 그 서사를 정직하게 정리하고 — 특히 **IAbilityCaster를 "계약"이 아니라 "마커"로 바로잡고** — 남은 로드맵을 순서대로 놓는다.

2026-08-19 갱신 — 이 문서 이후 바뀐 것
이 문서(2026-08-10) 작성 시점의 "브랜치 미머지·미검증"은 **해소됐다**: `ability-phase1-gating` → `main`
머지 완료(`b4b1295a`), **플레이 정상 동작 확인**. 로드맵 1번은 완료, 2번은 **축 불일치로 보류·철회**. 현재 상태의 단일 기준은 **STATUS.md**.

Part 1 · 서사 — 문제와 해결

포트폴리오의 강점은 기능의 양이 아니라 "분산·중복을 하나로 수렴시킨 문제해결"이다.

문제 — 전투 로직이 세 갈래로 흩어져 있었다

"공격을 발동한다"는 **같은 개념**이 캐스터마다 따로 구현돼 있었다.

- 보스** — `BossSkillController` 가 문자열 이름으로 캐스트, 스킬마다 클래스 하나(`GenesisSkill` 등). 실행 경로와 **이중 공존**.
- 플레이어** — `BaseCharacter` /유닛 12종에 `FireOneBullet` · `PerformAttackAction` 하드코딩. 평타·Q·E 발동 경로가 제각각.
- 일반몹** — `MonsterARController.RapidFireAsync` 등 컨트롤러마다 발사 로직이 직접 박힘.

축	이전 (분산·하드코딩)	현재 (통합 AbilitySystem)
공격 정의 위치	각 컨트롤러 메서드에 하드코딩	<code>AbilityData</code> (SO) + <code>EffectData[]</code>
캐스터별 경로	보스/플레이어/몹 3갈래 제각각	전부 <code>AbilityRunner</code> 단일 경로
새 공격 추가	새 클래스/메서드 작성 (프로그래머)	asset 조합 (디자이너 편집 가능)
스킬 단위	스킬 1개 = 클래스 1개 (상속)	스킬 = Effect 블록 조합
값(발수·간격)	코드 상수(<code>SerializeField</code>)	데이터 값 (<code>_count</code> / <code>_interval</code>)
관심사	계산·연출·데미지·발동이 한 메서드에	계산=Stat / 실행=Effect / 배달=투사체 / 발동=Runner

해결 — 하나의 데이터 주도 파이프라인



발동 진입점만 캐스터마다 다르고(입력·애니이벤트·BT·Timeline), 그 뒤는 전부 같은 파이프라인

핵심 설계 — 스킬 = "상속"이 아니라 "Effect 조합"

이전엔 스킬 하나가 클래스 하나였다. 지금은 스킬이 **Effect 블록의 조합**이다. 데미지+연출+상태이상을 붙이려면 클래스를 새로 짜는 게 아니라 **asset에 Effect를 엮는다**.

조합 예시 (코드 0줄)

```
"라이플 5연사" = RepeatEffect(5, 0.05s){ SpawnProjectiles(1), SpawnVFX(Muzzle) }
"보스 광역기" = DealDamage → SpawnVFX → AreaStrike(warning→delay→explode)
"랜덤 분기" = SelectRandomChoice → BranchByChoice{ A:소환, B:포격 }
```

관통하는 원칙: **값/로직/상태 3축 분류**(값 다름→asset 복제, 로직 다름→Effect 분리, 상태 있음→런타임 new) · **SO 불변**(런타임에 SO 필드 안 바꿈) · **투사체는 스냅샷 배달**(공격자가 죽어도 안전, 데미지는 '명중' 시점).

Part 2 · 정직화 — IAbilityCaster는 "계약"이 아니라 "마커"다

문서가 코드보다 반 발 앞서 있던 지점. 여기를 바로잡는 게 이번 점검의 핵심.

현재 코드의 진실

`public interface IAbilityCaster { }` — **멤버 0개의 빈 인터페이스**. 포트폴리오·README는 이것 "하나의 계약(TryUseAbility)"이라 부르지만, 실제로는 **아무것도 강제하지 않는 신원 표식(마커)**이다. 투사체는 아예 `Caster = null` 로 둔다.

원래 계획된 2가지 역할

계획	의도	실제로 어디로 갔나
① 공통 진입점 <code>TryUseAbility(int)</code>	모든 캐스터가 같은 시그니처를 노출 → 상위 계층이 캐스터 종류를 몰라도 호출	플레이어가 2축(슬롯+비트) + Commit/Fire 분리로 진화, 몸·보스와 진입점이 갈라짐 → 억지 공통 메서드는 누수 추상화 라 인터페이스에서 제거
② <code>TagContainer</code> 노출	게이팅 시 러너가 캐스터 태그를 인터페이스로 읽음	<code>AbilityContext.Tags</code> 로 이사. 캐스터 타입에 결합 안 되고, 캐스터 없는 투사체도 자연 처리

정직한 결론

지금의 빈 마커는 "미완성"이 아니라, 계획했던 두 책임이 각자 더 맞는 자리(`UseAbility`의 2축 구조 · `AbilityContext`)로 흡수되며 남은 껍데기다. 억지 공통 메서드를 강요하지 않은 성숙한 판단이다. 문제는 코드가 아니라 옛 서사를 안고 있는 문서다.

push(지금) vs pull(미래) — 왜 지금은 마커로 충분한가

밀어주기 · push 현재

발동 순간 확정된 값을 `ctx` 에 담아 건넨다.

```
ctx = { CasterGO, Stat, Object=총구, Tags, TargetPoint }
```

총구·스탯·태그처럼 발동 시점에 고정된 값에 적합. 현재 모든 Effect가 이걸로 충분.

물어보기 · pull 조건부 미래

실행 도중 캐스터에게 실시간으로 되묻는다.

```
ctx.Caster.GetAimDirection()
```

유도탄·실시간 락온·소환수 등록처럼 발동 후 변하는 값에 필요. 이걸 하려면 인터페이스에 멤버가 있어야 함.

지금 pull이 필요한 Effect는 0건이다. 조준은 `ctx.Object` (총구 회전), 타겟은 투사체 명중 판정, 데미지는 스냅샷으로 이미 다 풀렸다. 따라서 지금 인터페이스에 메서드를 넣는 건 아무도 안 쓰는 API를 미리 짓는 과잉설계다.

결정 (A)

마커로 유지한다. 코드는 그대로, 문서 서사만 "계약 = Runner + Data + IEffect, IAbilityCaster = 신원 마커"로 정직하게 수정. 로드맵에 B 승격 조건을 명시해 둔다: pull이 필요한 Effect가 실제로 등장할 때, 그 Effect가 요구하는 역할별 소계약을 하나씩 붙인다. (뚱뚱한 인터페이스 하나가 아니라

`IAimProvider` · `ISummonOwner` 처럼 쪼갬다 — 이미

`IChoiceHandler` · `ISpawnPointProvider` · `ISummonRegistry` 가 그 씨앗.)

Part 3 · 로드맵 — 순서대로

원칙: 넓이(Effect·캐릭터 추가)가 아니라 "동작 증명 → 깊이"로. 새로 짓기 전에 있는 걸 마감·검증·측정한다.

지금 0. 정직화 & 마감

노력: 작음 · 반나절

- 문서 서사 수정 — "계약(`IAbilityCaster` / `TryUseAbility`)" 표현을 "마커 + 실제 계약은 `Runner/Data/Effect`"로. 완료 2026-08-19 — README.md 반영
- 인코딩 UTF-8 통일 — `DealDamage.cs` · `BranchByChoice.cs` 등 CP949로 저장돼 주석이 깨진 파일 재저장. GitHub 브라우징 인상.
- 필드 가시성 정리 — `SpawnProjectiles` 의 `public` + 언더스코어 모순을 `[SerializeField]` `private` 로 통일.

완료 1. 컴파일 · 플레이 실측

2026-08-19 종료

Phase 1(게이팅 단일화)이 **main**에 머지되고 플레이로 검증됐다. "동작하는 걸 보여줄 수 있다"는 조건 충족.

- ☒ 에디터 컴파일 + 플레이 검증 — 이 구조가 적용된 상태로 정상 동작 확인.
- ☒ `ability-phase1-gating` → `main` 머지(`b4b1295a`). 죽은 심볼 잔여 참조 0 · 프리팹/씬 `CharacterCombat` 참조 0 · UI 폴링 배선 확인.
- II **Q_Skill** 클립 `AbilityBeat` 배선은 의도적 보류 — 쿨·궁게이지·글로우는 이벤트 없이 정상 동작하므로 게이팅 검증에 영향 없음. 연출 배선은 포트폴리오 가치가 낮다고 판단.

보류 2. 역할 전투 버티컬 슬라이스 + GC 측정

2026-08-19 재판단

철회 — 측정 측이 주장과 맞지 않는다

이 시스템의 명제는 "발동 3곳→1곳 · 스킬=조합 · 새 스킬 코드 0줄"로 **변경 비용**에 관한 것이라, 캐스트당 힙 할당은 그 명제를 검증하지 않는다. 게다가 아래 가설은 **사실이 아니다** —

`AbilityData.BuildRuntimeEffects()` 가 **캐스트마다** `new List<IEffect>` 를 판다. 측정하면 실제 병목도 아닌 곳을 포폴 숫자 때문에 캐싱 최적화하게 되므로 하지 않는다. (Week 1 배선은 2026-07-14 시점에 이미 되어 있었다.)

이미 만든 조각(`ApplyStatModifier` · `StatusRunner` · 역할/표식)을 새로 짓지 않고 배선·시연·측정. 대표작 #1을 보이는 데모로.

- 머니샷 — 브레이커 방깍(-Def) + 버퍼 버프(+Atk)가 **한 데미지 숫자로 합산**되고 각자 독립 만료되는 걸 화면/로그로. ("공격력+30% vs 방어력-40% → 최종 XXX")
- Before/After GIF — 딜러 단독 타격 vs 표식→버프 없는 타격의 데미지 점프.
- GC 실측 — `CreateRuntime()`⇒`this` (무상태 공유)로 **캐스트당 힙 할당 ≈ 0**임을 Profiler Before/After로. (에디터 과대계상 주의 → 상대 비교로 제시)

깊이(선택) 3. StatusSystem — 스택 정책 · 만료 · 안전 해제

노력: 큼 · 대표작으로 밀 때만

많은 포트폴리오가 빠뜨리는 "상태 지속 관리"의 정점. **tick 소유권**(대상이 독립 tick, Effect는 무상태) · **스택 정책 3종**(중첩/갱신/독립) · **대상 사망 시 안전 해제**(null-풀 반환 가드). "표준 구현"에서 이탈하는 깊이 축.

조건부

4. IAbilityCaster → 역할별 소계약으로 B 승격

트리거: pull-Effect 등장

지금 하지 않는다. 유도탄·실시간 락온·소환수 관리처럼 **실행 중 캐스터 질의가 실제로 필요한 Effect를 만들 때**, 그 Effect가 요구하는 계약(`IAimProvider` 등) **하나만** 인터페이스로 올린다. Effect는 `if (ctx.Caster is IAIMProvider aim)` 로 필요한 계약만 조회. "YAGNI로 마커에 머무르다, 필요가 증명되자 계약을 키웠다"가 **서사로도 더 강하다.**

한 줄 요약

방향(문제해결형 통합)은 실물이 받쳐주는 **강한 소재**.

2026-08-19 종결 — 정직화·검증은 끝났고 측정은 **측이 맞지 않아 철회했다**. **Ability** 트랙은 여기서 종료하며, 남은 항목(Phase 2 · CDR · B 승격 · StatusSystem)은 전부 **트리거 조건부다**. `IAbilityCaster` 는 **의도된 마커**로 두고, 승격 조건만 명시한다 — 그게 가장 성숙한 상태다.

Operation Kivotos · AbilitySystem · Narrative & Roadmap · 2026-08-10 (**2026-08-19 상태 갱신**) · 근거 문서: README.md · AbilitySystem_Portfolio.md · CharacterMigration.md · BossMigration.md · RoleSystemDesign.md · PortfolioEnhancement.md · Aug_Plan.md · Phase1_Result.md · 코드: Ability/*.cs, Data/Ability/**/*.cs